INTELIGENTNA ANALIZA DANYCH

WYKŁAD NR 3

Kamil Stokfiszewski

Spis treści

1. Trening nienadzorowany ze współzawodnictwem 2
2. Jednowarstwowa liniowa sieć samoorganizująca się (SOM) 3
   2.1 Algorytm treningu jednowarstwowej liniowej sieci samoorganizującej się 7
   2.2 Sieć SOM się do kompresji obrazów 9
   2.3 Przykład zastosowania sieci SOM w kompresji obrazów 13
      2.3.1 Specyfikacja zadania projektowego 15

1. Trening nienadzorowany ze współzawodnictwem

W niniejszym rozdziale zapoznamy się z metodą treningu sieci neuronowych poprzez tzw. trening nienadzorowany ze współzawodnictwem neuronów. Główne różnice pomiędzy metodami treningu nadzorowanego (które poznaliśmy w trakcie Zjazdów 1 oraz 2) są następujące:

Uczenie nadzorowane:

- wzorzec treningowy składa się z zarówno z sygnału wejściowego do sieci jak i oczekiwanej odpowiedzi sieci na tenże sygnał wejściowy,
- trening dąży do minimalizacji błędu pomiędzy odpowiedzią sieci a wartością oczekiwaną,
- sieci trenowane metodami z nadzorem stosowane są w aproksymacji sygnałów i klasyfikacji obiektów.

Uczenie nienadzorowane:

- wzorzec treningowy składa się tylko z sygnału wejściowego i nie zwiera informacji zwrotnej o oczekiwanej odpowiedzi sieci na tenże sygnał,
- metody uczenia nienadzorowanego prowadzą do samoorganizacji sieci czyli uporządkowania wektorów wagowych neuronów względem danych wejściowych,
- sieci podlegające treningowi nienadzorowanego służą do modelowania, klasteryzacji oraz klasyfikacji danych.

Istnieje wiele mechanizmów treningu nienadzorowanego i samoorganizacji sieci neuronowych, w tym między innymi, oparte o regułę asocjacji Hebba. W niniejszym rozdziale zajmiemy się jednak jednym z najbardziej popularnych mechanizmów treningu nienadzorowanego i samoorganizacji sieci, jakim jest mechanizm współzawodnictwa (konkurencji neuronów).

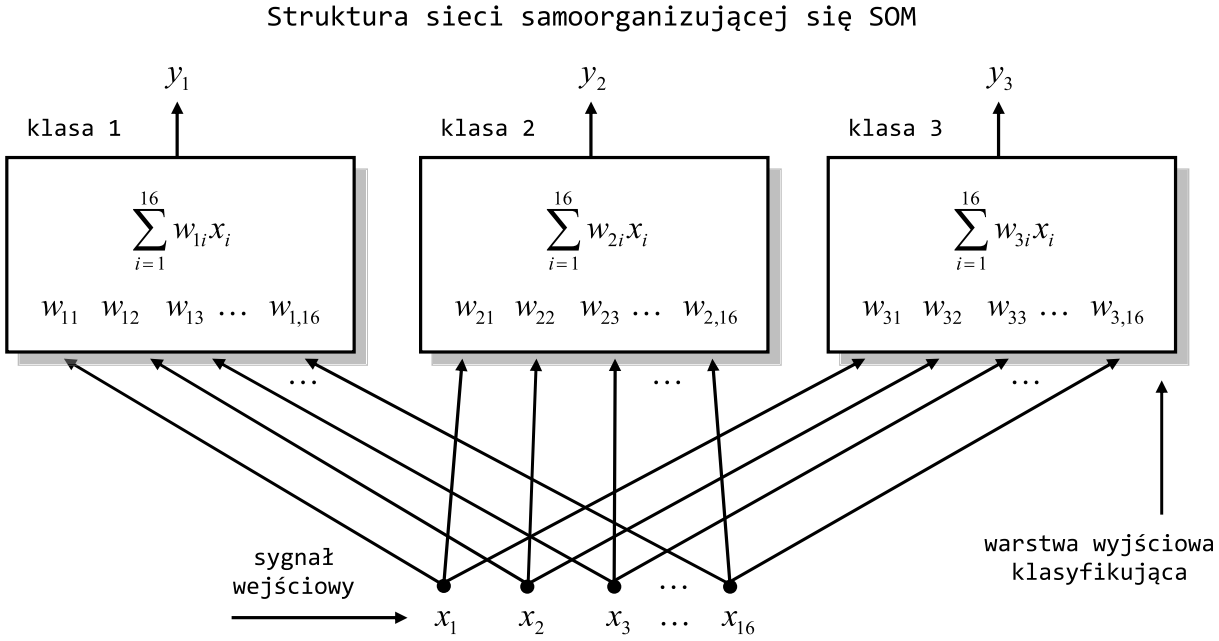
Samoorganizujące się sieci neuronowe służą najczęściej do:

- wykrywania istotnych cech w sygnale wejściowym, podobieństw i różnic pomiędzy różnymi klasami sygnałów wejściowych,
- modelowania rozkładu danych wejściowych
- analizy skupień, czyli klasteryzacji, wektorów o podobnych postaciach,
- analizy głównych składowych (ang. *principal component analysis*, w skrócie PCA), prowadzącej do redukcji wymiarowości danych wejściowych do cech najistotniejszych,
- prototypowania – wykrywania typowych reprezentantów pewnej grupy wektorów, które mogą posłużyć do kompresji, kodowania czy kwantyzacji sygnałów wejściowych.

Współzawodnictwo neuronów podczas treningu sieci oznacza, że neurony niejako konkurują ze sobą i adaptacja wag następuje jedynie w neuronie zwycięskim (dla metody „zwycięzca bierze wszystko” – ang. *winner takes all*, w skrócie WTA). W kolejnym rozdziale zajmiemy się konkretnym przypadkiem liniowej, jednowarstwowej sieci neuronowej trenowanej w sposób nienadzorowany ze współzawodnictwem neuronów w przykładowym zastosowaniu kompresji obrazów.

2. Jednowarstwowa liniowa sieć samoorganizująca się (SOM)

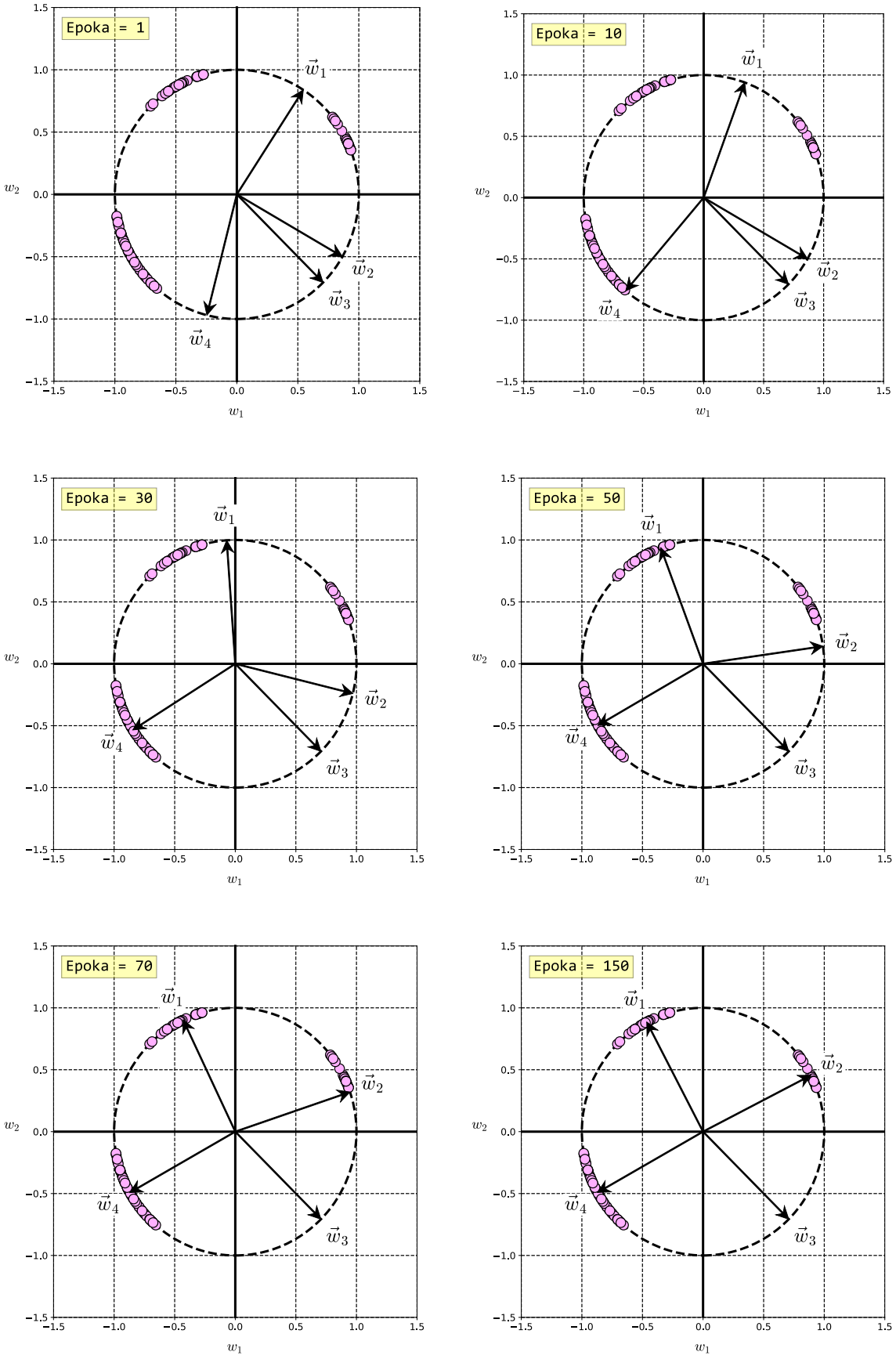
W niniejszym rozdziale zajmiemy się opisem działania samoorganizującej się jednowarstwowej, liniowej sieci neuronowej (zwanej siecią SOM – ang. *self organizing map*) trenowanej w sposób nienadzorowany ze współzawodnictwem neuronów. Architektura tej sieci jest identyczna z architekturą sieci MADALINE, omawianą w trakcie Zjazdów nr 1 oraz 2, jednak w sposób zasadniczy różnią się tu cele jak i metoda treningowa. Na Rys. 2.1 przedstawiono schemat rozważanej sieci neuronowej.



Rysunek 2.1. *Struktura sieci samoorganizującej się SOM.*

Zasada treningu nienadzorowanego sieci SOM polega na dostosowaniu wag uczonych neuronów do wektorów wejściowych w taki sposób, aby wektor wagowy pojedynczego neuronu jak najlepiej reprezentował pojedyncze skupisko (czyli klaster) wektorów wejściowych o podobnych do siebie współrzędnych obecny w sposób naturalny w danych treningowych właściwych dla danego zagadnienia rozwiązywanego siecią neuronową typu SOM.

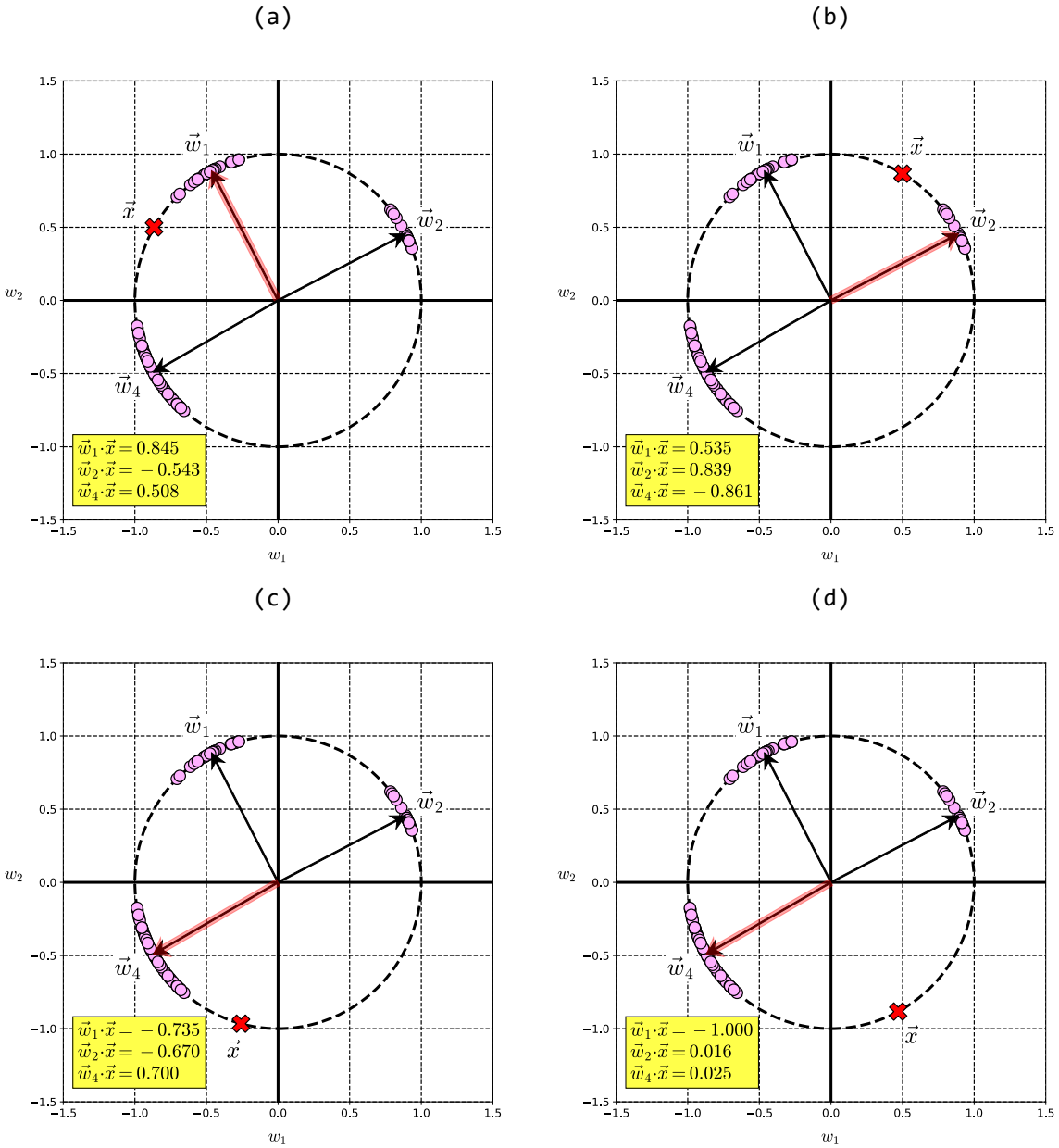
Podajmy przykład, który wyjaśni zasadę treningu samoorganizującej się sieci neuronowej. Na poszczególnych schematach Rys. 2.2 znajdują się kolejne etapy treningu sieci SOM. Jak widać, sieć reprezentowana 4-ma dwuwymiarowymi wektorami wagowymi $\boldsymbol{w}_k = [\, w_1^{(k)}, w_2^{(k)}]$, $k = 1, \ldots, 4$, a zatem składa się ona z 4 neuronów, z których każdy posiada 2-elementowy wektor wagowy. W epoce treningowej nr 1 wagi wszystkich neuronów ustawione są na znormalizowane wektory losowe. Następnie wektory wagowe poszczególnych neuronów sieci są modyfikowane iteracyjnie w procesie treningowym w taki sposób, aby zbliżyć je do skupisk wektorów obecnych w danych wejściowych (w podanym przykładzie można wyraźnie wyróżnić 3 takie skupiska, zwane także klastrami). W kolejnych epokach treningowych poszczególne wektory wagowe sieci, które znajdują się najbliżej wybranych klastrów obecnych w danych wejściowych, zbliżają się do nich coraz bardziej, aż do sytuacji w której dany wektor wagowy sieci staje się niejako średnim reprezentantem wektorów należących do odpowiadającego mu klastra. Taki jest właśnie cel treningu nienadzorowanego sieci SOM – uzyskać postaci wektorów wagowych, stanowiących średnie reprezentacje klastrów obecnych w danych wejściowych.



Rysunek 2.2. *Ustawienie wag neuronów sieci SOM w kolejnych epokach treningowych.*

Należy tu zauważyć, że z Rys. 2.2 wynika także, że zarówno wektory reprezentujące dane wejściowe jak i wagi sieci SOM są znormalizowane przez cały czas trwania procesu treningowego, w szczególności dotyczy to wektorów wagowych sieci (widać to poprzez fakt, iż wszystkie wspomniane wektory znajdują się na okręgu o jednostkowym promieniu, a więc ich długości są przez cały czas trwania procesu treningowego równe jednościom). Choć nie wynika to bezpośrednio z sekwencji schematów przedstawionych na Rys. 2.2, w pojedynczej epoce treningowej modyfikowana jest waga tylko jednego neuronu, tak zwanego *neuronu zwycięskiego* (ang. *winner neuron*), to znaczy ta spośród wag sieci, która we wspomnianej epoce treningowej znajduje się najbliżej pojedynczego wektora danych treningowych podawanego w tejże epoce na wejście sieci. Pozostałe wagi nie ulegają zmianie. Taki sposób modyfikacji wag w procedurze treningu nienadzorowanego określany jest mianem strategii *zwycięzca bierze wszystko* (ang. *winner takes all*, w skrócie strategii WTA). Z Rys. 2.2. wynika także, że efektem strategii WTA może być fakt, że niektóre z wag neuronów sieci SOM mogą nie zostać zmodyfikowane ani razu podczas jej uczenia i pozostać nie zmienione od początku, aż do samego końca procesu treningowego. Po zakończeniu treningu sieci neurony takie oczywiście nie reprezentują żadnego z potencjalnych klastrów wektorów znajdujących się w danych wejściowych i z tego powodu określane są często mianem *neuronów martwych* (ang. *dead neurons*). Neurony takie nie są przydatne podczas etapu działania sieci po zakończeniu procesu treningowego – z tego też powodu najczęściej są one usuwane z sieci po zakończeniu procesu nauczania. W sekwencji przykładowej z Rys. 2.2 neuronem martwym jest neuron posiadający wagę $\boldsymbol{w}_3$, która pozostała niezmieniona od początku, aż do samego końca procesu nauki trwającego w sumie 150 epok treningowych.

Przejdźmy teraz do wyjaśnienia sposobu działania sieci SOM. Po zakończeniu procesu treningowego i usunięciu martwych neuronów (w naszym przypadku jest to neuron o wadze $\boldsymbol{w}_3$) sieć jest gotowa do klasyfikacji sygnałów pojawiających się na jej wejściach. Sytuację tę pokazują schematy obecne na Rys. 2.3. Na Rys. 2.3 (a) wektor $\boldsymbol{x}$ oznaczony czerwonym krzyżykiem podany jest na wejście sieci o strukturze zobrazowanej na Rys. 2.1. Wszystkie neurony obliczają iloczyny skalarne swoich wektorów wagowych z wektorem wejściowym $\boldsymbol{x}$ i odpowiadają wartościami na wyjściach sieci (na Rys. 2.3 odpowiedzi te są podane w żółtych ramkach). Ponieważ wszystkie wektory – wejściowy i wagowe, biorące udział w omawianym procesie są znormalizowane, to, zgodnie z tym co omawialiśmy na Zjeździe nr 1, iloczyny skalarne wektorów wagowych poszczególnych neuronów z wektorem wejściowym $\boldsymbol{x}$ mogą być interpretowane jako miary podobieństwa pomiędzy wektorami wagowymi sieci, a podanym na jej wejście wektorem $\boldsymbol{x}$. W ten sposób wektor $\boldsymbol{x}$ jest niejako klasyfikowany jako przynależny do klastra obecnego w danych wejściowych, którego średnim reprezentantem jest najbliższy mu wektor wagowy. A zatem sieć dokonuje klasyfikacji nowych wektorów wejściowych do odpowiednich klastrów, które w sposób automatyczny zostały zidentyfikowane jako obecne w danych wejściowych podczas procesu treningu nienadzorowanego. Czerwone strzałki na Rys. 2.3 symbolizują wagi neuronów najbliższych kolejnym wektorom wejściowym $\boldsymbol{x}$ i jednocześnie identyfikują klasę sygnałów obecnych w danych wejściowych do której przynależą kolejne wektory wejściowe ze schematów (a)–(d) na Rys. 2.3. Warto jeszcze zwrócić uwagę na schemat (d) z Rys. 2.3. Obrazuje on sytuację, w której wektor wejściowy $\boldsymbol{x}$ został zaklasyfikowany jako przynależny do klastra reprezentowanego wagą $\boldsymbol{w}_4$ odpowiedniego neuronu sieci przykładowej. Jednocześnie empiryczne porównanie wizualne położenia wektora $\boldsymbol{x}$ i klastra reprezentowanego wagą $\boldsymbol{w}_4$ wskazuje, że taka kwalifikacja przynależności wektora $\boldsymbol{x}$ jest błędna, ponieważ wektor ten znajduje się daleko od przypisanego mu klastra. Zauważmy jednak, że iloczyny skalarne wszystkich wag sieci, łącznie z wagą $\boldsymbol{w}_4$ neuronu zwycięskiego, mają bardzo



Rysunek 2.3. *Klasyfikacja wektorów wejściowych przez wytrenowaną sieć neuronową typu SOM.*

małe wartości, co może wskazywać na potencjalny fakt wykrycia danej stanowiącej próbkę nowego klastra obecnego w danych wejściowych do sieci, bądź zidentyfikowanie danej błędnej, która powinna zostać odrzucona np. jako błąd pomiarowy obecny w danych wejściowych. W pierwszym przypadku należałoby dokonać dotrenowania sieci uwzględniając potencjalne pojawienie się w danych wejściowych wektorów tworzących nową klasę sygnałów wejściowych. W drugim przypadku sieć może posłużyć do wykrywania i identyfikacji błędnych lub/i niepełnych danych obecnych w sygnałach wejściowych.

Podajmy jeszcze konkretny przykład zastosowania sieci SOM. Załóżmy, że sklep internetowy oferujący książki i czasopisma z dziedzin technicznych i nauk przyrodniczych założył serwis internetowy oferujący wspomniane pozycje do zakupu. Z użyciem odpowiednich bibliotek (np. Google Analytics) oprogramowanie serwisu monitoruje parametry użytkowników wchodzących na strony serwisu, takich jak np. poszczególne podstrony z pozycjami książkowymi, które odwiedził dany użytkownik, czas jaki spędził na danej podstronie, zakupy pozycji jakich

dokonał korzystając z serwisu. Informacje te można odpowiednio przedstawić w postaci wektorowej (literatura wskazuje tu bardzo wiele skutecznych i sprawdzonych metod wektoryzacji danych internetowych). Następnie wektory te potraktować jak dane treningowe dla sieci SOM. Po zebraniu odpowiednio dużej liczby rekordów można spróbować wykorzystać tę sieć w celu odnalezienia klastrów reprezentujących poszczególne grupy klientów serwisu. Sieć powinna odnaleźć te klastry, a następnie, analizując zachowanie nowych użytkowników serwisu, może posłużyć do ich klasyfikacji do jednej z wcześniej odnalezionych automatycznie grup. Na bazie wspomnianej klasyfikacji można zaoferować nowemu użytkownikowi sugestię pozycji książkowych i czasopism, którymi mógłby być zainteresowany poprzez analizę wyborów dokonywanych przez użytkowników z grupy do której przynależy nowa osoba. Oczywiście taki sposób działania serwisu zwiększa potencjalnie jego rentowność poprzez reklamę celowaną produktów dedykowanych zidentyfikowanym grupom użytkowników. Zauważmy także, że w takim podejściu do klasyfikacji nie musimy dokonywać interpretacji uzyskanych klastrów. Tzn. właściciel sklepu internetowego nie musi dokonywać mapowania odnalezionych klastrów na rzeczywiste grupy społeczne użytkowników sklepu, np. matematyków, informatyków, fizyków czy inżynierów budownictwa. Sama informacja o przynależności do klastra, bez konieczności dokonywania jego interpretacji, pozwala na skuteczne zwiększenie rentowności sklepu. Nadal jednak taka interpretacja jest możliwa, a sieć typu SOM znacznie ułatwia jej dokonanie.

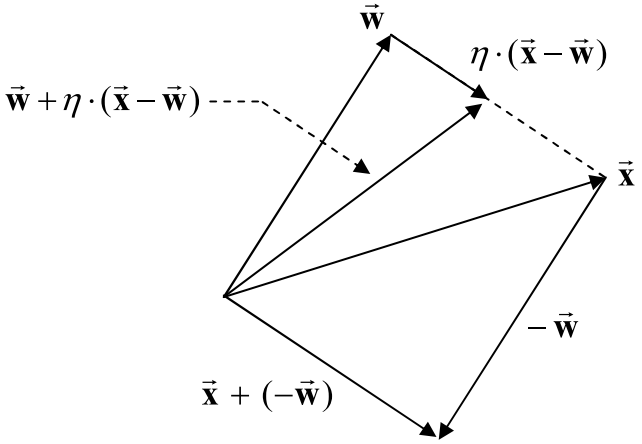
W tym miejscu zakończmy rozważania wstępne dotyczące zasady treningu i działania sieci typu SOM. W następnym rozdziale sformułujemy szczegółowo algorytmy treningu takich sieci oraz podamy przykład zastosowania sieci SOM w problemie kompresji obrazów.

2.1 Algorytm treningu jednowarstwowej liniowej sieci samoorganizującej się

W tym podpunkcie przedstawiony zostanie algorytm treningu jednowarstwowej liniowej sieci samoorganizującej się. Przedstawmy najpierw intuicyjny opis działania tego algorytmu. Tak jak w przypadku algorytmów treningowych poznanych wcześniej dla innych typów sieci neuronowych, uczenie sieci SOM podzielone jest na iteracje zwane epokami. W każdej epoce treningowej wszystkie wzorce uczące prezentowane są na wejścia sieci. Po prezentacji pojedynczego wzorca uczącego (czyli propagacji tego wzorca od wejść do wyjść sieci) dokonywana jest adaptacja wag sieci. W przypadku algorytmu treningu nienadzorowanego z mechanizmem WTA (*winner takes all*) współzawodnictwa neuronów, wybierany jest neuron zwycięski, czyli ten którego odpowiedź na wyjściu sieci była największa, i tylko jego waga podlega modyfikacji, zaś wagi pozostałych neuronów sieci nie są zmieniane. Modyfikacja wektora wagowego neuronu zwycięskiego polega na jego nieznacznym (zależnym od wartości współczynnika nauczania) zbliżeniu do wektora wejściowego, który spowodował jego aktywację. Proces ten powtarza się w pojedynczej epoce dla wszystkich wzorców wejściowych i skojarzonych z nimi neuronów zwycięskich. Modyfikacja wagi neuronu zwycięskiego odbywa się wg poniższego wzoru:

$$\boldsymbol{w}_{k+1} = \boldsymbol{w}_k + \eta \cdot (\boldsymbol{x} - \boldsymbol{w}_k) \tag{2.1}$$

gdzie $\boldsymbol{w}_{k+1}$ jest nowym wektorem wagowym neuronu zwycięskiego w $k+1$ iteracji pojedynczej epoki treningowej, $\boldsymbol{w}_k$ jest aktualną postacią tego wektora, $\boldsymbol{x}$ jest pojedynczym wektorem sygnału wejściowego sieci, zaś η to krok treningowy będący małą rzeczywistą liczbą dodatnią. Rys. 2.4 pokazuje intuicyjną interpretację wzoru modyfikacji wag w procedurze treningu nienadzorowanego.



Rysunek 2.4. *Interpretacja geometryczna wzoru modyfikacji wag w trakcie treningu sieci typu SOM.*

Jak widać z Rys. 2.4. nowy wektor wagowy $\boldsymbol{w} + \eta \cdot (\boldsymbol{x} - \boldsymbol{w})$ neuronu zwycięskiego przybliżył się do wektora wejściowego $\boldsymbol{x}$ w stopniu regulowanym przez wartość współczynnika treningowego η. Procedura ta, realizowana w pojedynczej epoce treningowej dla wszystkich wektorów wejściowych i odpowiadających im wektorów wagowych neuronów zwycięskich, powtarzana jest odpowiednio dużą liczbę razy równą liczbie epok treningowych sieci SOM. Ostateczny efekt jest taki, jaki można zaobserwować na ostatnim schemacie Rys. 2.2 (odpowiadającym epoce treningowej nr 150) – wagi poszczególnych neuronów sieci ustawione zostały w centrach klastrów obecnych w danych wejściowych. Przedstawiona wyżej intuicyjna analiza działania algorytmu treningowego dla sieci samoorganizującej się jest niewątpliwie bardzo przydatna w zrozumieniu idei działania tej sieci, teraz jednak warto podać precyzyjne sformułowanie wspomnianej metody treningowej. Jest ono podane niżej w postaci pseudokodu 2.1.

1) Ustaw wszystkie wagi sieci na wartości losowe z przedziału $[-1\ ,\ 1]$ zawartego w zbiorze liczb rzeczywistych, a następnie dokonaj normalizacji wszystkich wektorów wagowych.

2) Dla epoki treningowej $l = 1, \dots, L$ powtarzaj wykonywanie kroków od 3) do 7).

3) Dla wzorca treningowego $k = 1, \dots, K$ powtarzaj wykonywanie kroków od 4) do 7).

4) Ustaw znormalizowany wektor $\boldsymbol{x}_k = \left[\ x_1^{(k)}, x_2^{(k)}, \dots, x_N^{(k)}\ \right]$ k-tego wzorca uczącego na wejścia sieci.

5) Oblicz wyjścia neuronów warstwy wyjściowej sieci wg wzoru:

$$y_i = \sum_{j=1}^{N} w_{ij} x_j^{(k)} = w_{i1} x_1^{(k)} + w_{i2} x_2^{(k)} + w_{i3} x_3^{(k)} +, \dots, w_{iN} x_N^{(k)}\ ,\ \ i = 1, \dots, M. \tag{2.1.1}$$

6) Znajdź neuron zwycięski, tzn. taki którego odpowiedź y_i, $i = 1, \dots, M$ uzyskana w kroku 5) była najwyższa, oznacz jego indeks przez $m \in \{1, \dots, M\}$ i dokonaj modyfikacji jego współczynników wagowych wg wzoru:

$$w_{mj} = w_{mj} + \eta \cdot (\ x_j^{(k)} - w_{mj}\ )\ ,\ j = 1, \dots, N. \tag{2.1.2}$$

7) Dokonaj normalizacji wektora $\boldsymbol{w}_m$, a następnie wykonaj skok do punktu 3).

Algorytm 2.1*. Algorytm treningu sieci SOM metodą nienadzorowaną z rywalizacją neuronów.*

Jak widać postać pseudokodu przedstawionego w Alg. 2.1 jest stosunkowo prosta. Warto jeszcze zauważyć, że zarówno wektory uczące $\boldsymbol{x}_k, k = 1, \dots, K$ jak i wagowe $\boldsymbol{w}_i, k = 1, \dots, M$ są znormalizowane przez cały czas procesu nauki sieci – patrz krok 4) Alg. 2.1 dla wektorów uczących $\boldsymbol{x}_k$ oraz krok 7) tego algorytmu dla wektorów wagowych $\boldsymbol{w}_i$, przy czym normalizacji wektorów treningowych $\boldsymbol{x}_k$ należy dokonać jednokrotnie, przed rozpoczęciem procesu nauki sieci. Dzięki takiemu rozwiązaniu obliczenie podobieństw pomiędzy wektorami wejściowymi i wagowymi sieci SOM może zostać wykonane poprzez proste obliczenie wyjść sieci, gdyż iloczyn skalarny, który obliczany jest przez neurony warstwy wyjściowej, jest w tym wypadku dobrze określoną miarą podobieństwa pomiędzy wektorami (patrz materiały do Zjazd nr 1). Dla pełnej ścisłości przypomnijmy jeszcze metodę normalizacji dowolnego N-elementowego rzeczywistego wektora $\boldsymbol{x}$, mamy:

$$x_i' = \frac{x_i}{\| \boldsymbol{x} \|}, \ i = 1, \dots, N, \ \ \| \boldsymbol{x} \| = \sqrt{\sum_{i=1}^{N} x_i^2} \tag{2.2}$$

gdzie x_i' jest i-tą współrzędną znormalizowanego wektora wejściowego $\boldsymbol{x}'$, x_i jest i-tą współrzędną wektora oryginalnego $\boldsymbol{x}$, zaś $\| \boldsymbol{x} \|$ jest długością euklidesową ostatniego z wymienionych wektorów.

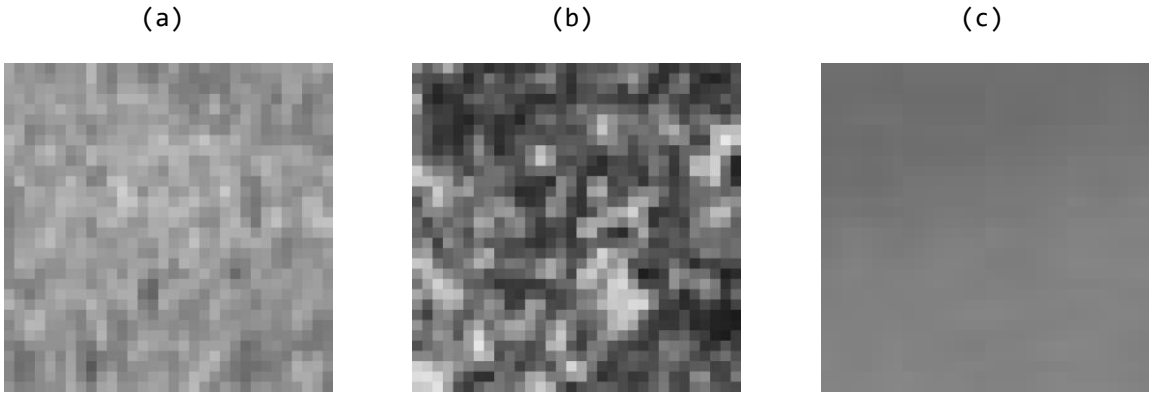
Po zakończeniu procesu treningu sieci SOM, identyfikacja przynależności wektora sygnału wejściowego do jednego ze znalezionych w trakcie nauki sieci klastrów, polega na prostym obliczeniu wyjść sieci i określeniu indeksu neuronu zwycięskiego. W tym przypadku również należy pamiętać o normalizacji wektora wejściowego przed podaniem go na wejścia sieci.

2.2 Sieć SOM się do kompresji obrazów

Niniejszy podrozdział opisuje zastosowanie sieci samoorganizującej się w zagadnieniu kompresji obrazów. Pomysł polega na podzieleniu kompresowanego obrazu na ramki o zadanym rozmiarze (np. 8x8 pikseli) i znalezieniu, przy użyciu sieci SOM, średnich reprezentantów ramek odzwierciedlających zawartość poszczególnych, charakterystycznych fragmentów obrazu. Np. dla obrazu z Rys. 2.5(a) przedstawiającego krajobraz z widokiem nieba, trawy oraz drzewa, wektory średnie, reprezentujące teksturę nieba, trawy oraz drzewa mogłyby wyglądać tak jak na Rys. 2.5(b)



Rysunek 2.5(a). *Przykładowy obraz kompresowany siecią SOM (źródło: https://jooinn.com/).*



Rysunek 2.5(b). *Przykładowe ramki średnie o rozmiarze 32x32 piksele, reprezentujące tekstury (a) trawy, (b) drzewa oraz (c) nieba, obecne w obrazie z Rys. 2.5(a).*

Następnie, przy użyciu znalezionych siecią SOM średnich reprezentantów ramek zawartych w kompresowanym obrazie, zakodowanie obrazu przy użyciu sekwencji indeksów neuronów zwycięskich reprezentujących poszczególne ramki, zamiast kodowania wartości poszczególnych pikseli, tak jak ma to miejsce w przypadku pliku z obrazem oryginalnym. Ponieważ zakodowanie indeksu neuronu zwycięskiego dla pojedynczej ramki obrazu wymaga użycia znacznie mniejszej liczby bitów niż zakodowanie wartości indywidualnych pikseli dla tejże ramki, osiągnięty efekt prowadzi do kompresji obrazu, czyli takiego jego zapisu w postaci pliku, którego rozmiar jest znacznie mniejszy niż rozmiar pliku oryginalnego. Odtworzenie skompresowanego obrazu polega na wstawieniu (odpowiednio przeskalowanych) ramek średnich reprezentowanych wagami neuronów sieci SOM w miejsce ramek oryginalnych. Oczywiście odtworzony obraz będzie obrazem o odpowiednio niższej jakości (zależnej w sposób proporcjonalny do współczynnika kompresji obrazu) w stosunku do jego oryginalnego odpowiednika, ponieważ podczas odtwarzania obrazu w miejsce każdej z oryginalnych ramek wstawiana jest jej średnia reprezentacja odnaleziona przy użyciu sieci SOM. Należy brać pod uwagę także fakty konieczności zapamiętania wektorów wagowych sieci reprezentujących ramki średnie oraz jasności poszczególnych ramek obrazu oryginalnego (tzn. euklidesowe długości wektorów ramek oryginalnych) w pliku skompresowanym, co oczywiście również ma (negatywny) wpływ na wartość współczynnika kompresji.

Opisany tu przykład zastosowania sieci SOM w kompresji obrazów jest dołączony do materiałów Zjazdu nr 3 w postaci pliku źródłowego języka `python`, o nazwie `som.py`. Przykład ten zostanie szczegółowo omówiony i zaimplementowany wspólnie podczas spotkania w ramach Zjazdu nr 3. Na schemacie Alg. 2.2 przedstawiono pseudokod opisanego wyżej algorytmu kompresji obrazu monochromatycznego z użyciem sieci SOM.

1) Wczytaj obraz wejściowy $\boldsymbol{X}$ w skali szarości o rozdzielczości $N \times N$ pikseli.

2) Podziel obraz na ramki o wymiarach $n \times n$ pikseli każda, dokonaj ich rozpłaszczenia i potraktuj każdą z nich jako n^2- elementowy wektor $\boldsymbol{x}_k,\ k = 1, \dots, \left(\frac{N}{n}\right)^2$.

3) Dokonaj normalizacji każdego z wektorów wejściowych $\boldsymbol{x}_k$ korzystając ze wzoru (2.2).

4) Utwórz sieć neuronową typu SOM o strukturze podanej na Rys. 2.1, mającą n^2 wejść oraz $K \in \mathbb{N}$ neuronów w warstwie wyjściowej, gdzie K jest empirycznym parametrem algorytmu.

5) Wybierz liczbę epok treningowych $M \in \mathbb{N}$ oraz współczynnik nauczania $\eta \in \mathbb{R}$ i dokonaj treningu sieci SOM na znormalizowanych wektorach wejściowych $\boldsymbol{x}_k$ stosując Alg. 2.1.

6) Po zakończeniu nauki usuń z sieci te neurony, których wagi nie zostały zmodfikowane ani razu podczas jej treningu (tzn. neurony martwe).

7) Rozpocznij etap kodowania obrazu. Dla każdej rozpłaszczonej, n^2- elementowej i znormalizowanej ramki obrazu $\boldsymbol{x}_k,\ k = 1, \dots, \left(\frac{N}{n}\right)^2$ ustaw tę ramkę na wejście sieci i oblicz odpowiedzi wszystkich neuronów sieci. Wybierz neuron zwycięski, tzn. taki którego odpowiedź była największa, i wstaw jego indeks do listy $\mathcal{L}$. Jednocześnie oblicz długość wektora $\boldsymbol{x}$ aktualnie przetwarzanej nieznormalizowanej ramki obrazu oryginalnego korzystając ze wzoru: $\|\boldsymbol{x}\| = \sqrt{\sum_{i=1}^{n^2} x_i^2}$, podziel ją przez współczynnik kwantyzacji $q \in \mathbb{N}$, który jest empirycznym parametrem algorytmu, dokonaj zaokrąglenia uzyskanego wyniku dzielenia do najbliższej liczby całkowitej i wstaw tę wartość do listy $\mathcal{T}$. Wykonaj powyższe kroki dla wszystkich ramek kodowanego obrazu.

8) Wstaw do listy $\mathcal{W}$ wektory wagowe kolejnych neuronów sieci.

9) Utworzone w punktach 7) oraz 8) listy $\mathcal{L}, \mathcal{T}$ oraz $\mathcal{W}$ stanowią dane w pełni opisujące zakodowany obraz.

10) Rozpocznij etap dekodowania obrazu. Dla każdego indeksu neuronu zwycięskiego k z listy $\mathcal{L}$ dokonaj przypisania zdekodowanego wektora ramki obrazu: $\boldsymbol{x}_k = q \cdot \mathcal{T}_k \cdot \mathcal{W}_k$**.** Dokonaj zwinięcia rozpłaszczonego wektora $\boldsymbol{x}_k$ z powrotem do postaci macierzy o wymiarze $n \times n$ – elementów i wstaw tę macierz jako zdekodowaną ramkę do obrazu odtworzonego. Wykonaj powyższe działania dla wszystkich indeksów znajdujących się w liście $\mathcal{L}$.

11) Otrzymany w punkcie 10) obraz $\boldsymbol{X}'$ jest zdekompresowaną wersją obrazu wejściowego $\boldsymbol{X}$.

12) Oblicz uzyskany współczynnik kompresji obrazu CR (ang. *compression ratio*) oraz współczynnik jakości obrazu odtworzonego $PSNR$ (szczytowy stosunek sygnału do szumu, ang. *peak signal to noise ratio*) wg wzorów, odpowiednio (2.3) oraz (2.4). Para współczynników CR oraz $PSNR$ uzyskanych w procesie kompresji obrazu z użyciem sieci SOM stanowi pełną charakterystykę efektywności tego procesu.

Algorytm 2.2*. Algorytm kompresji obrazu monochromatycznego realizowanej z użyciem sieci SOM.*

Pozostaje jeszcze uściślenie sposobu obliczania wartości wskaźników efektywności CR i $PSNR$ procesu kompresji obrazu z użyciem sieci SOM, które zostały przedstawione w punkcie 12) Alg. 2.2. Rozpocznijmy od definicji szczytowego stosunku sygnału do szumu (ang. *peak*

signal to noise ratio) jako współczynnika opisującego jakość obrazu odtworzonego. Jego wartość mierzona jest w *decybelach* (dB) i zdefiniowana w następujący sposób:

$$PSNR = 10 \cdot log_{10}\left(\frac{255^2}{MSE}\right),\ MSE = \frac{1}{N^2}\sum_{i=1}^{N}\sum_{i=1}^{N}(x_{i,j} - x'_{i,j})^2. \quad (2.3)$$

We wzorze (2.3) wielkość MSE (ang. *mean squared error*) jest błędem średniokwadratowym pomiędzy obrazem oryginalnym, którego wartości pikseli oznaczone są symbolem $x_{i,j},\ i,j = 1, \ldots, N$, a obrazem odtworzonym, którego piksele oznaczone są w równaniu (2.3) symbolem $x'_{i,j}$. Liczba 255 obecna w liczniku definicji miary $PSNR$ odpowiada maksymalnej wartości (czyli wartości szczytowej) jaką może przyjąć pojedynczy piksel obrazu monochromatycznego zapisanego w 8-bitowej skali szarości.

Obliczenie współczynnika kompresji CR (ang. *compression ratio*) rozpocznijmy od określenia rozmiaru obrazu oryginalnego. Ponieważ jest to obraz w 8-bitowej skali szarości, zapamiętanie pojedynczego piksela wymaga użycia 8 bitów. A zatem zapamiętanie całego obrazu wymaga użycia $8 \cdot N^2$ bitów, gdzie N jest jego rozdzielczością, pionową i poziomą zarazem (w naszych eksperymentach posługujemy się obrazami kwadratowymi, co nie zmniejsza ogólności podjętych rozważań).

Obraz skompresowany zapamiętany jest z użyciem trzech list – listy $\mathcal{W}$ przechowującej wartości wag wszystkich neuronów sieci zapisanych jako liczby zmiennopozycyjne (`float`), listy $\mathcal{L}$ zawierającej indeksy neuronów zwycięskich (liczby całkowite), których wektory wagowe służą do odtworzenia obrazu, oraz listy $\mathcal{T}$, która zawiera skwantowane (z użyciem całkowitego współczynnika kwantyzacji q – patrz punkt 7 oraz 10 Alg. 2.2) wartości całkowite kodujące długości poszczególnych wektorów ramek obrazu oryginalnego (odpowiadające *de facto* za kodowanie jasności ramek obrazu oryginalnego).

Liczba elementów listy $\mathcal{W}$ wynosi $K \cdot n^2$, gdzie K jest liczbą neuronów w sieci zaś n rozdzielczością w pikselach pojedynczej, kwadratowej ramki obrazu. Możemy przyjąć, że na zapamiętanie pojedynczej wartości wagowej z listy $\mathcal{W}$ potrzeba 8 bitów (choć w rzeczywistości liczby zmiennopozycyjne typu `float` zapamiętywane są za pomocą 32 bitów, to liczba 8 bitów w zupełności wystarczy, aby odpowiednio dokładnie zapamiętać wartości rzeczywiste z interesującego nas przedziału $[-1, 1]$, bez zauważalnej utraty jakości odtwarzanego obrazu). A zatem całkowita liczba bitów potrzebnych do zapamiętania wszystkich elementów listy $\mathcal{W}$ wynosi $N_{\mathcal{W}} = 8 \cdot K \cdot n^2$.

Liczba elementów listy $\mathcal{L}$ jest równa liczbie $n \times n$ – elementowych ramek zawartych w kodowanym obrazie, a zatem jest równa co do wartości $(N/n)^2$. Każdy element zawiera indeks neuronu zwycięskiego, czyli indeks wektora wagowego z listy $\mathcal{W}$, który służy do odtworzenia danej ramki obrazu. Maksymalna wartość tego indeksu wynosi $K - 1$, gdyż taki jest indeks ostatniego elementu listy $\mathcal{W}$. A zatem aby zapamiętać pojedynczy indeks zawarty w liście $\mathcal{L}$ potrzebujemy dokładnie $\lfloor log_2(K-1) \rfloor + 1$ bitów[1] (stosując naturalny kod binarny wykorzystywany do kodowania liczb całkowitych nieujemnych). Wobec tego na mocy przedstawionych tu rozważań, całkowita liczba bitów potrzebna do zapamiętania całej zawartości listy $\mathcal{L}$ wynosi $N_{\mathcal{L}} = (N/n)^2 \cdot (\lfloor log_2(K-1) \rfloor + 1)$.

Ostatnia z list, tzn. lista $\mathcal{T}$ zawiera liczby całkowite, będące skwantowanymi, z użyciem całkowitego współczynnika kwantyzacji q, długościami poszczególnych wektorów ramek obrazu oryginalnego. Liczba elementów tej listy, tak jak w przypadku listy $\mathcal{L}$, jest równa liczbie

[1] Funkcja $\lfloor x \rfloor$ (podłoga, ang. *floor*) oznacza największą liczbę całkowitą mniejszą bądź równą argumentowi rzeczywistemu x tej funkcji – oczywiście liczba bitów musi być liczbą całkowitą.

$n \times n$ – elementowych ramek zawartych w kodowanym obrazie, czyli $(N/n)^2$. Jeśli założymy stałą liczbę bitów potrzebnych do zakodowania pojedynczego elementu rozważanej listy $\mathcal{T}$, to będzie ona równa liczbie bitów koniecznych do zakodowania elementu tej listy o największej wartości. A zatem wspomniana liczba bitów wyniesie $\left\lfloor log_2 \left(\max_{0 \le k < (N/n)^2} \mathcal{T}_k \right) \right\rfloor + 1$. Wobec tego liczba bitów potrzebnych do zakodowania wszystkich elementów listy $\mathcal{T}$ będze miała następującą wartość: $N_{\mathcal{T}} = (N/n)^2 \cdot \left(\left\lfloor log_2 \left(\max_{0 \le k < (N/n)^2} \mathcal{T}_k \right) \right\rfloor + 1 \right)$.

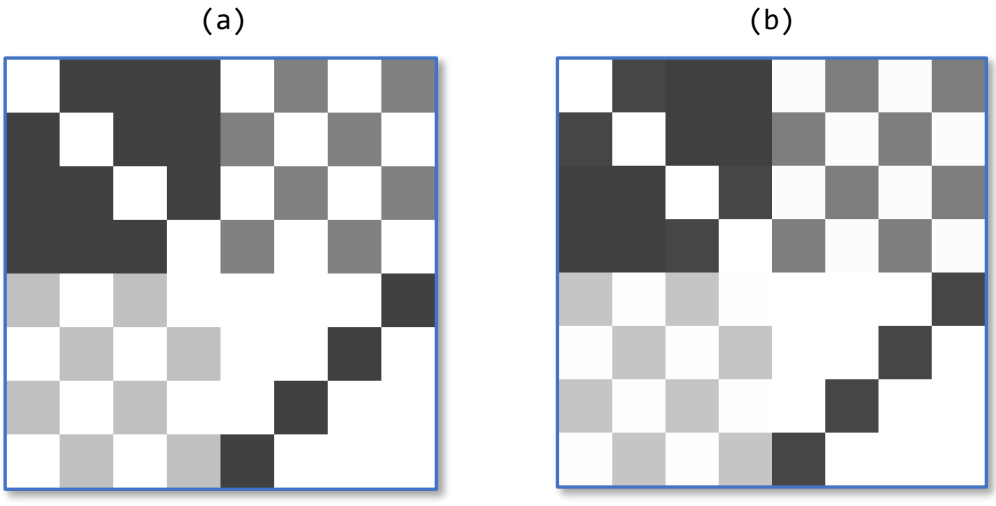
Zbierając wszystkie powyższe rozważania w całość i definiując współczynnik kompresji jako iloraz liczby bitów potrzebnych do zakodowania obrazu oryginalnego i liczby bitów potrzebnych do zapamiętania obrazu skompresowanego dostajemy:

$$CR = \frac{8\,N^2}{N_{\mathcal{W}} + N_{\mathcal{L}} + N_{\mathcal{T}}} = \frac{8\,N^2}{8\,K\,n^2 + (N/n)^2 \cdot \left(\lfloor log_2(K-1) \rfloor + \left\lfloor log_2 \left(\max_{0 \le k < (N/n)^2} \mathcal{T}_k \right) \right\rfloor + 2 \right)}, \qquad (2.4)$$

gdzie wszystkie symbole obecne we wzorze (2.4) zostały zdefiniowane w powyższych czterech paragrafach. Jak wspomniano wcześniej para współczynników CR oraz $PSNR$ daje pełną informację o efektywności procesu kompresji obrazu realizowanego przy użyciu sieci SOM.

2.3 Przykład zastosowania sieci SOM w kompresji obrazów

W niniejszym podrozdziale podamy konkretny przykład zastosowania sieci samoorganizującej się w kompresji obrazów. Wejściowy obraz eksperymentalny, który posłużył do testów metody kompresji obrazów z użyciem rozważanej sieci jest przedstawiony na Rys. 2.6 (a), zaś na Rys. 2.6 (b) znajduje się obraz skompresowany i odtworzony metodą podaną Alg. 2.2, której parametry zostaną omówione w dalszej części niniejszego podpunktu.



Rysunek 2.6. *Obraz eksperymentalny służący do testów metody kompresji obrazów z użyciem rozważanej sieci SOM, (a) obraz oryginalny, (b) obraz odtworzony po dokonaniu kompresji i dekodowania.*

Obraz na Rys. 2.6 jest obrazem o rozdzielczości 8x8 pikseli w 8-bitowej skali szarości. Po zastosowaniu programu eksperymentalnego realizującego kompresję obrazów z użyciem sieci SOM, dla rozmiaru ramki równego 2x2 piksele, 128 neuronów przed rozpoczęciem treningu

sieci, współczynniku nauczania η równym 0.2, współczynniku kwantyzacji q równym 10 i liczbie epok równej 100 otrzymano obraz zdekodowany widoczny na Rys. 2.6 (b) oraz następujący raport końcowy dotyczący procesu treningu, kompresji i dekodowania obrazu:

```
resolution             = 8 x 8
frame_size             = 2 x 2

num_frames             = 16
num_neurons            = 4
num_weights_per_neuron = 4
max_quantized_length   = 51

bits_per_index         = 2
bits_per_length        = 6
bits_per_weight        = 5

org_size_bits          = 512
cmp_size_bits          = 208

w_1 = [ 0.425  0.565  0.565  0.425]
w_2 = [ 0.500  0.500  0.500  0.500]
w_3 = [ 0.686  0.172  0.172  0.686]
w_4 = [ 0.632  0.317  0.317  0.632]

CR   =  2.462
PSNR = 37.545 dB
```

Raport stwierdza, że liczba neuronów pozostałych po nauce sieci wyniosła 4 (z początkowych 128 neuronów – usunięte neurony były neuronami martwymi), liczba bitów na pojedynczy indeks w liście $\mathcal{L}$ wyniosła 2, liczba bitów na pojedynczą wartość w tablicy $\mathcal{T}$ wyniosła 6 (co wynika z maksymalnej wartości obecnej w tej tablicy równej 51) oraz liczba bitów na pojedynczą wagę w tablicy $\mathcal{W}$ wyniosła 5. To w konsekwencji spowodowało, że rozmiar obrazu skompresowanego równał się 208 bitom, co w porównaniu z 512 bitami potrzebnymi do zapamiętania obrazu oryginalnego dało ostatecznie współczynnik kompresji o wartości bliskiej 2.5. Jednocześnie miara jakości PSNR obrazu odtworzonego po kompresji wyniosła około 37.5 dB, co jak widać z Rys 2.6 (a) i (b) spowodowało, że obraz odtworzony jest praktycznie nie do odróżnienia od obrazu oryginalnego. Dodatkowo wyświetlone zostały wartości wag wszystkich czterech neuronów pozostałych po nauce sieci. Wagi te kodują *de facto* tekstury o rozmiarze ramki 2x2 obecne w obrazie.

Podanym tu przykładem zakończymy rozważania dotyczące wykorzystania sieci SOM w problemie kompresji obrazów niniejszym. Pozostaje jeszcze podać propozycję zadania projektowego służącego systematycznemu zbadaniu zdolności sieci SOM w rozważanym problemie.

2.3.1 Specyfikacja zadania projektowego

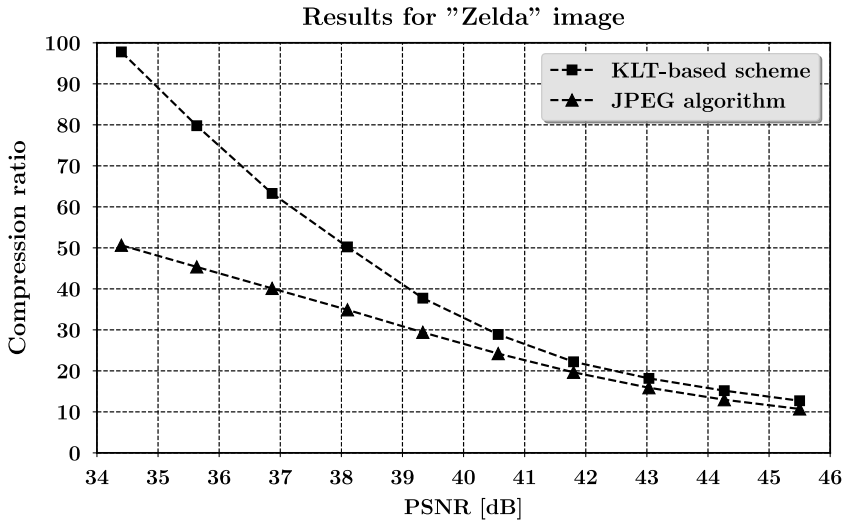
Na podstawie materiałów podanych w Rozdziale 2 należy:

1. Stworzyć skrypt w języku Python, implementujący i testujący efektywność sieci samoorganizującej się w problemie kompresji obrazów.

2. Do testów należy wybrać jeden (lub więcej dla osób zainteresowanych) z obrazów przykładowych obecnych w materiałach do kursu, są to pliki `zelda.bmp`, `cameraman.bmp` oraz `peppers.bmp`, zawierające obrazy w skali szarości o rozdzielczości 128x128 pikseli.

3. Dla rozmiaru ramki równego 2x2 piksele dobrać odpowiednio parametry nauki (sugerowane wartości to: liczba epok treningowych równa 20, stała kwantyzacji q równa 10 oraz współczynnik nauczania η równy 0.2) i zbadać wpływ początkowej liczby neuronów na charakterystykę efektywnościową sieci, to znaczy wartości parametrów CR i PSNR.

4. Dla początkowych liczb neuronów równych, odpowiednio 4, 8, 16, 32, 64, 128 i 256 odbyć proces treningu, kompresji i dekodowania obrazu z użyciem zaimplementowanej sieci SOM i na podstawie zebranych w ten sposób danych wykreślić charakterystykę zależności pomiędzy stopniem kompresji CR, a jakością odtworzonego obrazu PSNR po dekompresji, podobną do tej uwidocznionej na Rys. 2.7.[1]



Rysunek 2.7. *Przykładowa charakterystyka jakościowa porównująca dwie metody kompresji obrazów – metodę JPEG oraz metodę opartą o tzw. przekształcenie Karhunena-Loeve-go dla obrazu `zelda` o rozdzielczości 512x512 pikseli zawierająca zależność pomiędzy stopniem kompresji CR, a jakością odtworzonego obrazu zdekompresowanego PSNR.*

5. Osoby chętne i zainteresowane tematem mogą przeprowadzić testy badające wpływ innych parametrów metody – rozmiaru ramki, stałej kwantyzacji czy rozdzielczości obrazu na charakterystykę efektywnościową metody kompresji obrazów z użyciem sieci SOM.

[1] Wartości podane w charakterystyce przykładowej dotyczą zupełnie innych niż proponowana, specjalistycznych metod kompresji obrazów i nie należy się nimi sugerować podczas opracowywania wyników własnych, które będą z pewnością znacznie odbiegały od efektywności uzyskiwanej przez metody dedykowane.